



Cypher is Turing-Complete

A Formal Proof via 2-Counter Machine Simulation

Pierre Halftermeyer · Neo4j

March 2026

Abstract. We prove that Cypher 25, the graph query language of Neo4j, is Turing-complete. The proof proceeds by showing that a single `RETURN` statement using `reduce()`, `CASE` expressions, and list comprehensions can simulate any 2-counter machine (Minsky machine). Since 2-counter machines are known to be Turing-complete, the result follows by reduction. We address the bounded-step objection via two complementary resolutions and verify the construction by executing the simulation on a live Neo4j Aura instance.

§1 The Claim

Cypher 25, restricted to a single RETURN statement with `reduce()`, list comprehensions, and CASE expressions, **is Turing-complete**.

§1.1 Proof Strategy

We prove Turing-completeness by *reduction*:

Turing Machine $\xrightarrow{\text{Minsky 1967}}$ 2-Counter Machine $\xrightarrow{\text{this proof}}$ Cypher `reduce()`

- 2-counter machines are known to be Turing-complete [1].
- We show Cypher can simulate *any* 2-counter machine.
- Therefore Cypher is Turing-complete.

§2 2-Counter Machines

§2.1 Definition

A **2-counter machine** $M = (Q, q_0, q_{\text{halt}}, \delta)$ consists of:

Component	Meaning
Q	Finite set of states
$q_0 \in Q$	Initial state
$q_{\text{halt}} \in Q$	Halting state
$\delta : Q \rightarrow \text{Instructions}$	Transition function

Each instruction is one of:

- $\text{INC}(c, q')$ — increment counter $c \in \{A, B\}$, go to state q'
- $\text{JZDEC}(c, q_z, q_p)$ — if $c = 0$ go to q_z , else decrement c and go to q_p

Fact (Minsky 1967)

For any Turing machine T , there exists a 2-counter machine M that simulates T . Two-counter machines are the *simplest* known Turing-complete model: only two unbounded integers, only two instruction types, and a finite-state controller. If Cypher can simulate this, it can simulate *anything*.

§2.2 Example: A 4-State Program

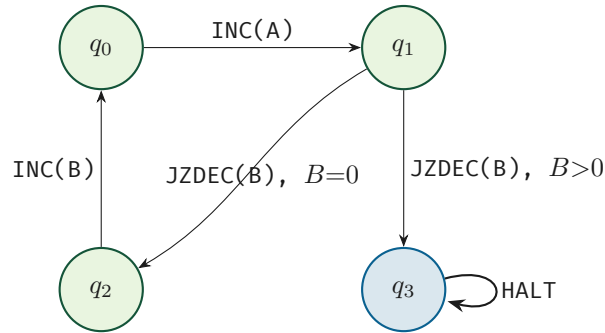


Figure 1 — A 4-state 2-counter machine that increments A , tests B , loops or halts.

§3 Encoding in Cypher

§3.1 Program Encoding

Encode the transition function δ as a Cypher **list of maps**:

```
LET program = [
  {state: 0, op: 'INC', counter: 'A', next: 1},
  {state: 1, op: 'JZDEC', counter: 'B', q_zero: 2, q_pos: 3},
  {state: 2, op: 'INC', counter: 'B', next: 0},
  {state: 3, op: 'HALT', counter: '', next: 3}
]
```

This is a *finite, static* list — no graph writes needed. `program[i]` gives $\delta(q_i)$ in $O(1)$.

§3.2 Machine State

The full machine configuration (q, c_A, c_B) is a single Cypher **map**:

```
{state: 0, A: 0, B: 0}
```

Conventions: `state = -1` means *halted*; counters are non-negative integers; the initial configuration is `{state: 0, A: 0, B: 0}`.

§4 The Simulation

§4.1 Core `reduce()` Implementation

Each iteration of `reduce()` performs **one machine step**:

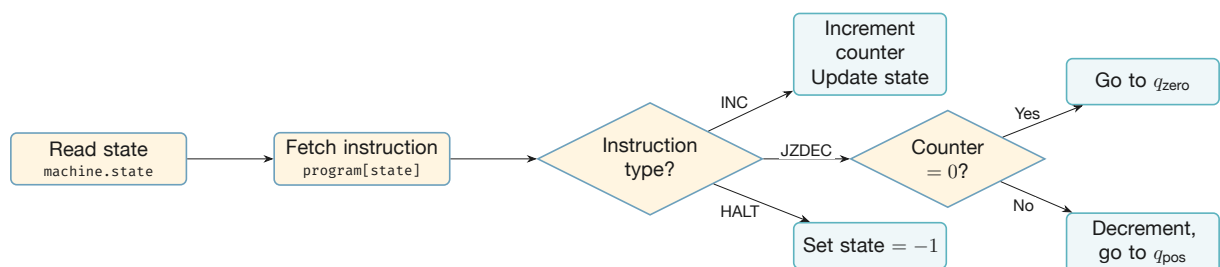


Figure 2 — Execution flow of one `reduce()` iteration.

```
CYPHER 25
LET program = [ /* ... as above ... */ ]
LET max_steps = 1000000 // arbitrary bound (see §5)

LET result = reduce(
  machine = {state: 0, A: 0, B: 0},
  step IN range(1, max_steps) |
  CASE WHEN machine.state = -1
    THEN machine // halted: identity
  ELSE
    head([instr IN [program[machine.state]] |
      CASE instr.op
        WHEN 'INC' THEN
          CASE instr.counter
            WHEN 'A' THEN
              {state: instr.next,
               A: machine.A + 1, B: machine.B}
            WHEN 'B' THEN
              {state: instr.next,
               A: machine.A, B: machine.B + 1}
          END
        WHEN 'JZDEC' THEN
          CASE instr.counter
            WHEN 'A' THEN
              CASE WHEN machine.A = 0
                THEN {state: instr.q_zero,
                     A: 0, B: machine.B}
                ELSE {state: instr.q_pos,
                     A: machine.A - 1, B: machine.B}
              END
            WHEN 'B' THEN
              CASE WHEN machine.B = 0
                THEN {state: instr.q_zero,
                     A: machine.A, B: 0}
                ELSE {state: instr.q_pos,
                     A: machine.A, B: machine.B - 1}
              END
            END
          END
        WHEN 'HALT' THEN
          {state: -1, A: machine.A, B: machine.B}
        END
      ])
    )
  RETURN result
```

§4.2 Key Idiom: Let-Binding in `reduce()`

LET is *not available* inside `reduce()`. The workaround uses a single-element list comprehension:

```
// Let-binding idiom:
head([instr IN [program[machine.state]] |
  // 'instr' is now bound -- use it freely
  CASE instr.op WHEN 'INC' THEN ... END
])
```

This wraps the expression in a single-element list [expr], the comprehension *binds* the variable name, and head() unwraps the result.

§5 Correctness Proof

After k iterations of reduce(), the value of machine equals the configuration of machine M after k steps.

Base case ($k = 0$):

machine = {state : 0, A : 0, B : 0} = initial configuration of M ✓

Inductive step. Assume after k iterations: machine = {state : q_k , A : a_k , B : b_k } matches M 's configuration.

At iteration $k+1$, the reduce() body:

1. Looks up program[machine.state] → retrieves instruction $\delta(q_k)$.
2. Executes via CASE:

Instruction	Cypher returns	Matches?
INC(A, q')	{state : q' , A : $a_k + 1$, B : b_k }	✓
JZDEC(A, q_z, q_p), $a_k = 0$	{state : q_z , A : 0, B : b_k }	✓
JZDEC(A, q_z, q_p), $a_k > 0$	{state : q_p , A : $a_k - 1$, B : b_k }	✓
HALT	{state : -1, ...} — identity thereafter	✓

Symmetric cases for counter B . Therefore machine after $k+1$ iterations matches M 's configuration after $k+1$ steps. □

§6 The Bounded-Step Objection

The reduce() proof uses range(1, max_steps) — a *finite* bound. Strict Turing-completeness requires *unbounded* computation.

Approach	Mechanism	Achieves
§6.1	IN TRANSACTIONS	Loop up to 2^{63} steps — practically unbounded
§6.2	Sufficiency argument	Set bound = halting time — universal for total functions

§6.1 Resolution via IN TRANSACTIONS

```
CYPHER 25
CREATE (:Machine {state: 0, A: 0, B: 0});

UNWIND range(1, 9223372036854775807) AS step // 2^63 - 1
CALL (step) {
  MATCH (m:Machine)
  // 1/0 guard BEFORE list indexing
  WITH m,
    CASE WHEN m.state = -1 THEN 1/0
    ELSE $program[m.state]
    END AS instr
  SET m.state = CASE instr.op
  WHEN 'INC' THEN instr.next
  WHEN 'JZDEC' THEN CASE instr.counter
    WHEN 'A' THEN
      CASE WHEN m.A = 0
        THEN instr.q_zero ELSE instr.q_pos END
    WHEN 'B' THEN
      CASE WHEN m.B = 0
        THEN instr.q_zero ELSE instr.q_pos END
    END
  WHEN 'HALT' THEN -1 END,
  m.A = CASE
  WHEN instr.op = 'INC' AND instr.counter = 'A'
    THEN m.A + 1
  WHEN instr.op = 'JZDEC' AND instr.counter = 'A'
    AND m.A > 0 THEN m.A - 1
  ELSE m.A END,
  m.B = CASE
  WHEN instr.op = 'INC' AND instr.counter = 'B'
    THEN m.B + 1
  WHEN instr.op = 'JZDEC' AND instr.counter = 'B'
    AND m.B > 0 THEN m.B - 1
  ELSE m.B END
} IN TRANSACTIONS OF 1 ROW
ON ERROR BREAK
```

Termination mechanism. When the machine reaches the HALT instruction, state is set to -1. On the *next* iteration, the guard CASE WHEN m.state = -1 THEN 1/0 fires *before* any list indexing, triggering a division-by-zero error caught by ON ERROR BREAK, which cleanly exits the loop. This ensures \$program[-1] is never evaluated.

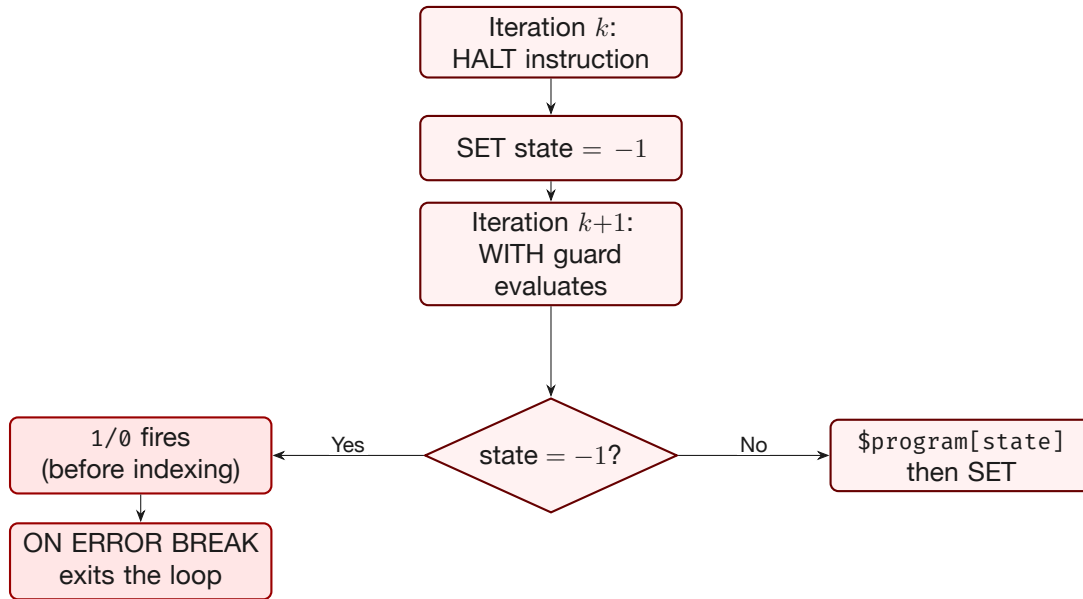


Figure 3 — Termination mechanism: the 1/0 guard fires before `$program[-1]` is ever evaluated.

Note on the 2^{63} bound. Strictly speaking, `range(1,  $2^{63}-1$ )` is still a finite bound, so this does not achieve *theoretical* unbounded computation. True unbounded execution would require an external driver (e.g., a client-side loop that restarts the query if it hasn't halted). In practice, $2^{63} \approx 9.2 \times 10^{18}$ steps exceeds any physically realizable computation.

§6.2 Resolution via Sufficiency Argument

For any Turing machine T on input w that halts in $f(|w|)$ steps:

$$\text{max_steps} = f(|w|) \implies \text{reduce}() \text{ correctly simulates } T \text{ on } w$$

Since:

- Every **decidable** language L has a decider T_L that halts on all inputs.
- For each T_L , there exists a computable bound f .
- Cypher `reduce()` with `range(1, f(|w|))` correctly decides L .

This shows Cypher is **computationally universal for total functions**. The gap with full Turing-completeness concerns only non-halting computations (addressed by §6.1).

§7 Required Primitives

The proof uses only:

Primitive	Role in the proof
<code>reduce()</code>	Fold over a list — provides iteration
<code>CASE WHEN</code>	Conditional branching
Integer <code>+1, -1, = 0</code>	Counter operations
<code>Map {state, A, B}</code>	Machine state representation
List indexing <code>program[i]</code>	$O(1)$ instruction lookup
<code>head([v IN [expr] ...])</code>	Let-binding (convenience only)

No graph operations. No APOC. No GDS. Pure Cypher expressions suffice.

§8 Verified Execution on Neo4j Aura

Both simulation approaches were tested on a Neo4j Aura instance running Neo4j 5.x with Cypher 25 support.

§8.1 Test Program: Manual Trace

The 4-instruction program from §3.1:

Step	Instruction	State	A	B
0	INC(A, q_1)	q_0	$0 \rightarrow 1$	0
1	JZDEC(B, q_2, q_3); $B=0$	q_1	1	0
2	INC(B, q_0)	q_2	1	$0 \rightarrow 1$
3	INC(A, q_1)	q_0	$1 \rightarrow 2$	1
4	JZDEC(B, q_2, q_3); $B>0$	q_1	2	$1 \rightarrow 0$
5	HALT	q_3	2	0

Expected result: {state: -1, A: 2, B: 0}

§8.2 Results

Result: reduce() simulation

{A: 2, B: 0, state: -1} ✓

Result: IN TRANSACTIONS simulation

m.state = -1, m.A = 2, m.B = 0 ✓

Both approaches produce the expected result, matching the manual trace step-for-step.

References

- [1] Minsky, M. L. (1967). *Computation: Finite and Infinite Machines*. Prentice-Hall.
- [2] Neo4j Documentation: [Cypher Manual](#).
- [3] Advent of Code Cypher Solutions (2015–2025) — 150+ solved algorithmic problems.
github.com/halftermeyer